1. Software Requirements Specification

FUNCTIONAL

Document Control	
Author(s)	Michael Gardner
Document Version	4.1.0
Applies to FUNCTIONAL	^4.1
Status	Released
Last Updated	2026-04-27
SPDX-License-Identifier	BSD-3-Clause
License File	See the LICENSE file in the project root
Copyright	© 2026 Michael Gardner, A Bit of Help, Inc.

Project Profile	
Variant	library
Library Role	utility
Application Role	N/A
Assurance	spark-targeted
Execution	sequential
Parallelism	none
Platform	desktop, server, embedded
Execution Environment	linux, macos, windows, ada-runtime
Processor Architecture	amd64, arm64, arm32
Deployment	native

Table of Contents

1. Software Requirements Specification 1
2. Introduction 1
   2.1. Purpose 1
   2.2. Scope 1
   2.3. Definitions and Acronyms 1
   2.4. References 1
3. Overall Description 1
   3.1. Product Perspective 1
   3.2. Product Features 2
   3.3. User Classes 2
   3.4. Operating Environment 2
   3.5. Constraints 2
4. Interface Requirements 3
   4.1. User Interfaces 3
   4.2. Software Interfaces 3
      4.2.1. Ada API 3
   4.3. Hardware Interfaces 3
5. Functional Requirements 3
   5.1. Result Module (FR-R) 3
   5.2. Option Module (FR-O) 3
   5.3. Either Module (FR-E) 4
   5.4. Try Module (FR-T) 4
   5.5. Scoped Module (FR-S) 5
6. Quality and Cross-Cutting Requirements 5
   6.1. Performance (NFR-01) 5
   6.2. Reliability (NFR-02) 5
   6.3. Portability (NFR-03) 5
   6.4. Maintainability (NFR-04) 5
   6.5. Usability (NFR-05) 6
   6.6. Platform Abstraction (NFR-06) 6
   6.7. SPARK Formal Verification (NFR-07) 6
   6.8. Testability (NFR-08) 6
7. Design and Implementation Constraints 6
   7.1. System Requirements 6
      7.1.1. Hardware Requirements 6
      7.1.2. Software Requirements 6
8. Verification and Traceability 7
   8.1. Verification Methods 7
   8.2. Traceability Matrix 7
9. Appendices 8
   9.1. Change History 8

2. Introduction

2.1. Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for **Functional**, a type-safe functional programming library for Ada 2022.

2.2. Scope

Functional provides:

- Result type for explicit success/failure handling.
- Option type for null-safe optional values.
- Either type for neutral disjunction without error semantics.
- Try bridges for declarative exception-to-Result/Option conversion.
- Scoped guards for RAII resource management.

2.3. Definitions and Acronyms

Term	Definition
Discriminated Union	Ada record type with a discriminant controlling the variant.
Either	A discriminated union representing one of two valid values (Left or Right).
Functor	Abstraction for transforming wrapped values (Map).
Monad	Abstraction for sequencing computations (Map, And_Then).
Option	A discriminated union representing presence (Some) or absence (None).
RAII	Resource Acquisition Is Initialization — automatic cleanup pattern using Ada finalization.
Railway-Oriented Programming	A pattern where operations are chained, with errors automatically propagating through the pipeline.
Result	A discriminated union representing success (Ok) or failure (Error).
SPARK	A formally verifiable subset of Ada.

2.4. References

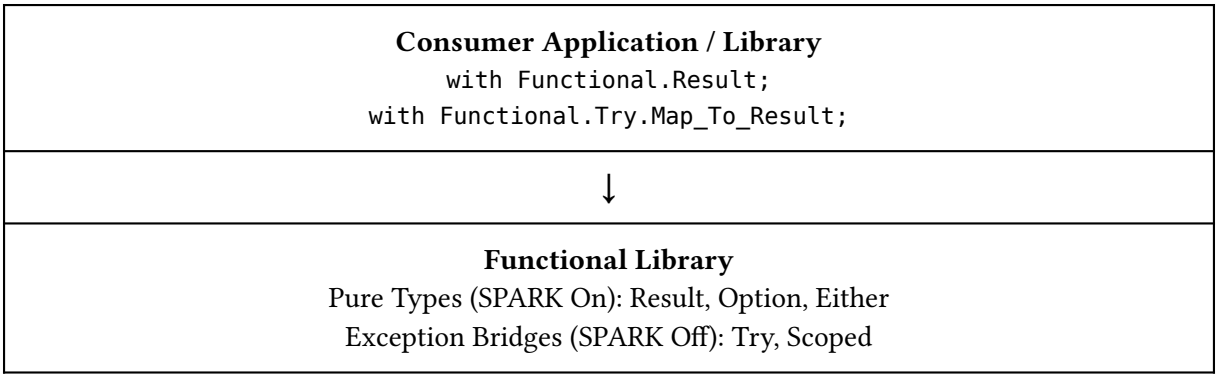
- Ada 2022 Language Reference Manual (ISO/IEC 8652:2023).
- SPARK 2014 Reference Manual.

3. Overall Description

3.1. Product Perspective

Functional is a standalone Ada 2022 library designed to enable functional programming patterns in Ada applications. It integrates with the Ada ecosystem through Alire package management and is consumed by projects such as **Hybrid_Lib_Ada**, **Hybrid_App_Ada**, and all downstream Ada projects in this organization.

Unlike the typical hybrid DDD/Clean/Hexagonal project, Functional does not have a layered architecture. It is a flat utility library organized by module (Result, Option, Either, Try, Scoped), with a SPARK boundary separating pure functional types from exception-handling infrastructure.



3.2. Product Features

1. **Result[T, E]**: Type-safe success/failure handling via discriminated union.
2. **Option[T]**: Null-safe optional values via discriminated union.
3. **Either[L, R]**: Neutral disjunction without error semantics.
4. **Try**: Declarative exception boundary adapters (Map_To_Result, Map_To_Result_With_Param, To_Result, To_Option).
5. **Scoped**: RAII guards for automatic resource cleanup using Ada finalization.

3.3. User Classes

User Class	Description
Domain Developers	Developers implementing business logic with Result/Option.
Infrastructure Developers	Developers implementing I/O boundaries with Try bridges.
Library Consumers	Ada developers using Functional types in their projects.

3.4. Operating Environment

Requirement	Specification
Platforms	desktop, server, embedded
Execution Environment	linux, macos, windows, ada-runtime
Processor Architecture	amd64, arm64, arm32
Ada Version	Ada 2022
Compiler	GNAT FSF 14+ or GNAT Pro
Package Manager	Alire 2.0+
SPARK Prover	GNATprove 14+ (optional, for verification)

3.5. Constraints

Constraint	Rationale
Ada 2022	Required for modern language features.
No Heap Allocation	Stack-based discriminated unions ensure embedded safety.
No Runtime Dependencies	Only the Ada standard library is permitted.
SPARK Subset	Core types must remain formally verifiable.

4. Interface Requirements

4.1. User Interfaces

None. This is a library, not an application.

4.2. Software Interfaces

Interface	Description
Ada.Exceptions	Used by the Try module for exception identity and message extraction.
Ada.Finalization	Used by the Scoped module for RAII guard finalization.

4.2.1. Ada API

```
with Functional.Result;
with Functional.Try.Map_To_Result_With_Param;

--  Instantiate with project-specific types
package My_Result is new Functional.Result (My_Type, Error_Kind);
```

4.3. Hardware Interfaces

None. The library is hardware-agnostic.

5. Functional Requirements

5.1. Result Module (FR-R)

Priority: Must **Description:** Provide a type-safe success/failure discriminated union.

ID	Requirement	Priority
FR-R01	Provide Ok constructor to create success values.	Must
FR-R02	Provide Error constructor to create failure values.	Must
FR-R03	Provide Is_Ok/Is_Error discriminant checks.	Must
FR-R04	Provide Value accessor with precondition Is_Ok.	Must
FR-R05	Provide Get_Error accessor with precondition Is_Error.	Must
FR-R06	Provide Map to transform success values.	Must
FR-R07	Provide Map_Error to transform error values.	Must
FR-R08	Provide And_Then for chaining fallible operations.	Must
FR-R09	Provide Or_Else for error recovery.	Must
FR-R10	Provide Unwrap_Or for default extraction.	Must
FR-R11	Provide Match for pattern matching.	Should
FR-R12	Provide Recover for error-to-success conversion.	Should

Acceptance Criteria:

- Result is exactly one of Ok(value) or Error(error).
- Value extraction is type-safe via discriminant checks.
- No exceptions are raised for expected error conditions.

5.2. Option Module (FR-O)

Priority: Must **Description:** Provide a null-safe optional value discriminated union.

ID	Requirement	Priority
FR-O01	Provide Some constructor for present values.	Must
FR-O02	Provide None constructor for absent values.	Must
FR-O03	Provide Is_Some/Is_None discriminant checks.	Must
FR-O04	Provide Value accessor with precondition Is_Some.	Must
FR-O05	Provide Map to transform present values.	Must
FR-O06	Provide And_Then for chaining optional operations.	Must
FR-O07	Provide Or_Else for absence recovery.	Must
FR-O08	Provide Unwrap_Or for default extraction.	Must
FR-O09	Provide Filter to conditionally keep values.	Should
FR-O10	Provide Match for pattern matching.	Should

Acceptance Criteria:

- Option eliminates null references.
- Absent values produce no runtime error on access through safe combinators.

5.3. Either Module (FR-E)

Priority: Must **Description:** Provide a neutral disjunction discriminated union.

ID	Requirement	Priority
FR-E01	Provide Left constructor.	Must
FR-E02	Provide Right constructor.	Must
FR-E03	Provide Is_Left/Is_Right discriminant checks.	Must
FR-E04	Provide Left_Value accessor with precondition Is_Left.	Must
FR-E05	Provide Right_Value accessor with precondition Is_Right.	Must
FR-E06	Provide Map_Left to transform left values.	Must
FR-E07	Provide Map_Right to transform right values.	Must
FR-E08	Provide Fold for consuming either side.	Should

Acceptance Criteria:

- Neither side is designated as “error” (distinguishes Either from Result).
- Useful for union types, parsing, and validation branches.

5.4. Try Module (FR-T)

Priority: Must **Description:** Provide declarative exception-to-Result/Option bridges for architectural boundaries.

ID	Requirement	Priority
FR-T01	Provide Map_To_Result for declarative exception mapping.	Must
FR-T02	Provide Map_To_Result_With_Param for parameterized actions.	Must
FR-T03	Support empty mappings for catch-all behavior.	Must
FR-T04	Support multiple exception-to-error mappings.	Must
FR-T05	Provide To_Result child package for procedural mapping.	Should
FR-T06	Provide To_Option child package for probe operations.	Should

ID	Requirement	Priority
FR-T07	Include exception message in error construction.	Must

Acceptance Criteria:

- Exception mapping uses data (mapping arrays) rather than procedural code.
- All exceptions at boundaries are caught and converted to Result or Option values.
- No exceptions escape through the Try boundary.

5.5. Scoped Module (FR-S)

Priority: Must **Description:** Provide RAII guards for automatic resource cleanup.

ID	Requirement	Priority
FR-S01	Provide Guard_For for unconditional RAII.	Must
FR-S02	Provide Conditional_Guard_For for conditional RAII.	Must
FR-S03	Catch and suppress exceptions from Release.	Must
FR-S04	Support user-defined Release procedures.	Must

Acceptance Criteria:

- Cleanup always executes, even when the guarded block raises an exception.
- Exceptions from Release are suppressed to prevent double-exception scenarios.

6. Quality and Cross-Cutting Requirements

6.1. Performance (NFR-01)

ID	Requirement
NFR-01.1	Zero heap allocation in core operations.
NFR-01.2	Use static dispatch (no tagged types in core).

6.2. Reliability (NFR-02)

ID	Requirement
NFR-02.1	Handle all exceptions at Try boundaries without leaking.
NFR-02.2	Do not propagate exceptions from Scoped guards.

6.3. Portability (NFR-03)

ID	Requirement
NFR-03.1	Compile on GNAT 14+ across Linux, macOS, and Windows.
NFR-03.2	Contain no platform-specific code in the library.

6.4. Maintainability (NFR-04)

ID	Requirement
NFR-04.1	Achieve at least 90% test coverage.
NFR-04.2	Maintain zero compiler warnings.

6.5. Usability (NFR-05)

ID	Requirement
NFR-05.1	Provide clear error messages from all operations.
NFR-05.2	Provide comprehensive documentation and usage examples.

6.6. Platform Abstraction (NFR-06)

ID	Requirement
NFR-06.1	Isolate exception handling to the Try module.
NFR-06.2	Isolate finalization to the Scoped module.

6.7. SPARK Formal Verification (NFR-07)

ID	Requirement
NFR-07.1	Verify core types (Result, Option, Either) with SPARK_Mode On.
NFR-07.2	Mark Try and Scoped with SPARK_Mode Off.
NFR-07.3	Provide instantiation tests for SPARK compatibility.

Module	SPARK_Mode	Rationale
Either	On	Pure functional type, no side effects.
Option	On	Pure functional type, no side effects.
Result	On	Pure functional type, no side effects.
Scoped	Off	Ada.Finalization.
Try	Off	Exception handling.

6.8. Testability (NFR-08)

ID	Requirement
NFR-08.1	Support unit testing of all public operations.
NFR-08.2	Provide SPARK instantiation tests for compatibility verification.

7. Design and Implementation Constraints

7.1. System Requirements

7.1.1. Hardware Requirements

No specific hardware requirements. The library operates within standard Ada runtime constraints.

7.1.2. Software Requirements

Category	Requirement
Compiler	GNAT FSF 14+ or GNAT Pro (Ada 2022 support).
Package Manager	Alire 2.0+.
SPARK Prover	GNATprove 14+ (optional, for formal verification).

8. Verification and Traceability

8.1. Verification Methods

Method	Description
Code Review	All changes reviewed before merge.
Integration Testing	Try with real exception scenarios.
SPARK Proof	Result, Option, Either core operations.
Unit Testing	All public operations of Result, Option, Either, Try, Scoped.

8.2. Traceability Matrix

Requirement	Module	Test
FR-R01..R12	Functional.Result	test_result.adb
FR-O01..O10	Functional.Option	test_option.adb
FR-E01..E08	Functional.Either	test_either.adb
FR-T01..T07	Functional.Try.*	test_try*.adb
FR-S01..S04	Functional.Scoped	test_scoped.adb
NFR-07	SPARK instantiations	spark_instantiations.ads

9. Appendices

9.1. Change History

Version	Date	Author	Changes
4.1.0	2025-12-18	Michael Gardner	Deprecate procedural Try functions; keep Option-based Try functions. Migrate to Typst format.
4.0.0	2025-12-12	Michael Gardner	Remove Expect functions; add Map_To_Result and Map_To_Result_With_Param packages.
3.0.0	2025-11-30	Michael Gardner	Initial architecture with Result, Option, Either, Try, and Scoped modules.